Module 03 — AI in Healthcare

Research and Technical Design Report

Project: HealthLens AI — Explainable Symptom-Language Analysis
Implementation choice: Option 2, Python + NLP with a Gradio interface
Student: José Miguel Tuozzo Izaguirre
Course: ITAI-2372 — Artificial Intel Applications
Professor: Srikanth Bhakthan
Assignment: Module 03 — AI in Healthcare
Date: October 12, 2026
Intended users: Student demonstrators and classmates; **not patients**

Abstract

HealthLens AI is a reproducible natural language processing (NLP) application designed to illustrate how a healthcare-related text interface can analyze fictional symptom statements without pretending to be a licensed diagnostic or triage service. The prototype applies text preprocessing and term-frequency/inverse-document-frequency (TF-IDF) feature extraction to 56 synthetic training sentences, then uses cosine similarity to assign one of seven **symptom-language topics**. A separate rule layer identifies a limited set of explicit warning phrases before topic selection, and an abstention rule declines out-of-scope statements. The interface displays its matching rationale and stresses that similarity is neither probability nor clinical confidence. The project is supported by systematic-review evidence regarding the limitations of existing symptom checkers and by WHO, MedlinePlus, and FDA material relating to patient safety and responsible health AI [1–4]. A 26-case synthetic functional evaluation and Python unit tests illustrate software behavior, not clinical effectiveness.

1. Innovation selected and problem statement

This project selects a recent, active healthcare AI problem: **assisting with interpretation of patient-reported natural-language symptom descriptions**. Most people describe symptoms conversationally rather than in standardized medical coding systems. An NLP interface can transform ordinary descriptions into structured categories that are easier to search, organize, or review. The innovation in HealthLens is not that it invents a new medical algorithm; it is an **explainable, safety-scoped implementation** of a familiar health NLP workflow appropriate for a student project.

There is substantial reason not to treat symptom checkers as authoritative clinicians. In a 2023 systematic review, Riboli-Sasco and colleagues analyzed 14 eligible studies and found highly variable accuracy across symptom checker products. Nine of 13 studies assessing triage reported suboptimal accuracy, leading the authors to call for more robust validation [1]. Rather than duplicate the assignment's example code, which assigns categories using a few term overlaps, the prototype adds a learned TF-IDF representation, an explicit warning override, input constraints, abstention, reproducible testing, and transparent limitations. **This prototype is not intended for clinical use.**

2. Research objectives

The technical goals are to (1) turn free-text English inputs into vector representations; (2) select the closest topic from a small, inspectable set of fictional scenarios; (3) demonstrate explicit warning-sign detection as an upstream safety mechanism; (4) handle selected negated warnings; (5) abstain when input falls outside the toy corpus; (6) report the basis of each decision; and (7) evaluate expected behavior against previously specified fictional cases. The educational goals are to demonstrate data quality, feature engineering, error analysis, explainability, safe interface design, and the boundary between a software test and a clinical validation study.

3. Data provenance and preprocessing

The training set is a local JSON file of 56 **hand-authored synthetic examples** across seven topic groups: respiratory, headache, digestive, skin, musculoskeletal, urinary, and eye/nose irritation. Eight sentences per topic provide elementary linguistic variation. The 26 evaluation cases are stored separately, also fictitious, and include topic examples, warning phrases, simple negations, irrelevant text, and an asymptomatic statement. The texts were not scraped from clinical resources, copied from copyrighted health content, collected from real patients, or taken from medical records. The evaluation labels are developer expectations, not physician annotations.

Text preprocessing starts with Unicode NFKC normalization, apostrophe normalization, and collapsing repeated white-space. `TfidfVectorizer` then converts text to lowercase, strips accent marks, tokenizes words, removes standard English stop words, and constructs unigram and bigram term features. Sublinear TF scaling reduces the influence of repeated terms. The inverse-document-frequency weights emerge from the synthetic corpus at runtime. The resulting sparse vectors are compared by **cosine similarity**. This demonstrates a practical data-to-features-to-comparison NLP pipeline without relying on a large language model. No NLTK model downloads are necessary. Linguistic variants not captured by the learned vocabulary are a limitation, and no separate lemmatization is performed.

4. Architecture and inference

The application's `app.py` contains only presentation logic and safe rendering; `healthlens/engine.py` contains the underlying analytical steps. The flow is **input → normalization → warning phrase rules → no-symptom/scope check → TF-IDF vector → cosine comparison → topic or abstention → explanation**. A warning overrides the statistical topic result. The trained vocabulary comes exclusively from the packaged synthetic JSON data and is reconstructed locally at startup. Real user input is not used for training.

Warning phrases were written for this educational exercise as a small illustrative set covering language associated with serious complaints. They are **not comprehensive or clinically certified**. A limited negation check recognizes constructs such as `no chest pain`; a clause boundary prevents that negation from suppressing a separate warning such as `but I cannot breathe`. When no positive warning is found, recognized negated warning phrases are removed from the text sent to TF-IDF so that "no chest pain" does not distort a muscle-discomfort topic match. This heuristic can fail under more complex grammar, multilingual text, or implied symptoms. Consequently, **absence of a trigger never means absence of danger**.

If there is no recognized warning, the model searches its synthetic text examples. The closest example's topic is returned only when cosine similarity is at least **0.20**. That value is a toy engineering cutoff, not clinically optimized or a probability. Inputs containing plainly asymptomatic expressions receive an out-of-scope response, and very long or empty inputs abstain. A classifier result shows the nearest training sentence, shared terms, and similarity score to support error analysis.

5. Interface and functional tests

The browser interface uses Gradio with one text area, example scenarios, an analysis button, a result panel, and a methods disclosure. Warning, topic, and abstention states are visually distinct. User-provided text is not inserted as raw HTML: the result renderer escapes displayed generated content. The app launches on localhost by default (`127.0.0.1`) and does not open a public sharing tunnel.

Unit tests are executed using `python -m pytest -q`. They cover explicit warnings, simple negations, warnings after a contrast word, precedence over model retrieval, similarity traceability, irrelevant input, output constraints, and data categories. `python evaluate.py` summarizes the independent synthetic fixtures. The evaluation is deliberately not compared with real triage decisions, and it does not report clinical diagnosis accuracy.

6. Evaluation and findings

The prototype was locally tested on **26 fixed synthetic cases**: eight expected explicit warnings, sixteen expected topic categories, and two expected abstentions. In this toy benchmark, all eight warning cases were detected, all eighteen other cases avoided a warning label, all sixteen expected topics matched, and both unknown cases abstained. These are **100% scores on carefully constructed fictional examples only**. They are particularly vulnerable to selection bias: the author wrote both the test scenarios and the application rules/examples, the dataset is tiny, the vocabulary is similar, and the cases have no independently established clinical ground truth.

The correct scientific interpretation is narrow: the implementation correctly follows its stated requirements for this small test suite. Nothing in these numbers establishes safety, generalization, medical utility, or performance in populations. Independent clinician-adjudicated evaluation is a future prerequisite, not an accomplished feature.

7. Ethical analysis and practical limitations

Safety. Incorrectly reassuring a person about symptoms can delay care. For this reason, no result claims that a condition is mild, that the person is safe, or that they should avoid clinical assessment. Warning detection is transparent but incomplete. A negative result does not replace medical advice. MedlinePlus provides background

context about recognized emergencies and professional help [2]; its content was not used as a dataset or fed into the model.

Fairness. Seven English-language topic clusters cannot represent real clinical diversity. The corpus omits slang, misspellings, many cultural idioms, disability-related communication patterns, older adults, pregnant people, children, rare diseases, coexisting symptoms, and all non-English users. This is a **significant generalizability problem**, not a minor inconvenience. Demographic disparities were not measured because no real cohorts or demographic labels were collected.

Privacy. The code intentionally has no patient database, data retention procedure, health account, third-party model API, or user analytics tracker. Nevertheless, a hosting platform could log IP addresses or other request metadata. These choices do not automatically constitute HIPAA compliance. A public deployment handling personal health information would need documented access controls, retention limits, audit processes, consent analysis, and legal/privacy review.

Explainability and automation bias. A similarity score can look numerical and therefore authoritative even when it is not clinically meaningful. HealthLens explicitly labels cosine similarity as a vector-space text statistic and shows the matched synthetic example. The WHO emphasizes risks arising from misinformation, bias, privacy failures, and automation bias in health AI [3]. Explainability and caution help students see those risks, but do not eliminate them.

Regulatory limits. Software function, intended use, and user population affect whether healthcare AI might be regulated as a medical device. The FDA's January 2026 Clinical Decision Support Software guidance describes this context [4]. This project makes no claim of FDA clearance or regulatory exemption; deployment as patient-facing decision software would require a distinct regulatory assessment.

AI assistance statement. ChatGPT was used to assist with application design, Python code generation, synthetic example and test-case creation, debugging, and report drafting, which this assignment permits. The submitted code and research documentation should be reviewed and demonstrated by the student. The application does not call ChatGPT or another large language model during symptom-language analysis.

8. Next steps

Future academic improvements could include properly consented, de-identified, expert-annotated vignettes; independently labeled test data; systematic error analysis; synonym handling; contextual negation models; calibrated abstention experiments; subgroup fairness measurements; and clinician-led review. A real health application additionally requires independent external validation, prospective safety studies, security and privacy engineering, deployment monitoring, and a clear human oversight process. No such work has been performed for this prototype.

9. Conclusion

HealthLens AI meets the educational goal of creating an end-to-end NLP application related to healthcare: it ingests text, computes data-driven features, applies interpretable rules, returns an explained result in a working web UI, and supports repeatable tests. Its most important design decision is **not to confuse symptom-language classification with medicine**. The student can demonstrate the technique, inspect every model decision, and discuss concrete safety and ethical risks without making unsupported clinical claims.

References

[1] Riboli-Sasco, E., El-Osta, A., Alaa, A., Webber, I., Karki, M., El Asmar, M. L., Purohit, K., Painter, A., & Hayhoe, B. (2023). Triage and diagnostic accuracy of online symptom checkers: Systematic review. *Journal of Medical Internet Research, 25*, e43803. https://doi.org/10.2196/43803

[2] U.S. National Library of Medicine / MedlinePlus. (2025, January 8 review). *Recognizing medical emergencies.* https://medlineplus.gov/ency/article/001927.htm

[3] World Health Organization. (2024, January 18). *WHO releases AI ethics and governance guidance for large multi-modal models.* https://www.who.int/news/item/18-01-2024-who-releases-ai-ethics-and-governance-guidance-for-large-multi-modal-models

[4] U.S. Food and Drug Administration. (2026, January). *Clinical decision support software: Guidance for industry and FDA staff.* https://www.fda.gov/regulatory-information/search-fda-guidance-documents/clinical-decision-support-software